

Xử lý ảnh số và Thị giác máy tính

Chương 1: OpenCV, numPY

Giảng viên: TS. Phạm Văn Tính

Email: pvtinh@hcmuaf.edu.vn

Khoa Công nghệ thông tin, Đại học Nông Lâm TP. HCM

Nội dung

- Tổng quan về OpenCV
- Tổng quan về numPy
- Một số ví dụ dùng openCV, numPy trong Python

Tổng quan về OpenCV

- OpenCV (Open Computer Vision library) là một thư viện mã nguồn mở về thị giác máy với hơn 500 hàm và hơn 2500 các thuật toán đã được tối ưu về xử lý ảnh, và các vấn đề liên quan tới thị giác máy tính. OpenCV được thiết kế một cách tối ưu, sử dụng tối đa sức mạnh của các dòng chip đa lõi... để thực hiện các phép tính toán trong thời gian thực, nghĩa là tốc độ đáp ứng của nó có thể đủ nhanh cho các ứng dụng thông thường
- Thư viện OpenCV – gồm khoảng 500 hàm – được viết bằng ngôn ngữ lập trình C và được thiết kế để tương thích với các hệ điều hành khác nhau (cross-platform) như Windows, Linux, MacOS... đóng vai trò xác lập chuẩn giao tiếp, dữ liệu, thuật toán cho lĩnh vực Computer Vision và tạo điều kiện cho mọi người tham gia nghiên cứu và phát triển ứng dụng

Tổng quan về OpenCV

- OpenCV được phát triển bởi Intel vào năm 1999 bởi Gary Bradski và phát hành phiên bản đầu tiên vào năm 2000. Mục tiêu ban đầu của OpenCV là nâng cao hiệu suất của các ứng dụng chuyên sâu CPU, cũng như tạo ra một thư viện thị giác máy tính.
- Năm 2005, OpenCV được sử dụng trong Stanley, chiếc xe giành chiến thắng trong cuộc thi DARPA Grand Challenge, đây là bước đệm hoàn hảo cho việc phát triển OpenCV trong tương lai.
- Năm 2006, phiên bản 1.0 của OpenCV được phát hành, bao gồm hơn 500 thuật toán và cấu trúc dữ liệu cơ bản cho thị giác máy tính.
- Năm 2009, phiên bản 2.0 của OpenCV được phát hành, mang lại nhiều thay đổi lớn cho giao diện C++, nhằm mục đích tạo ra các mẫu thiết kế dễ sử dụng và an toàn hơn, cũng như cải thiện hiệu suất và tính năng của các thuật toán.

Tổng quan về OpenCV

- Năm 2011, OpenCV bắt đầu hỗ trợ tăng tốc GPU cho các hoạt động theo thời gian thực.
- Năm 2012, OpenCV được quản lý bởi tổ chức phi lợi nhuận OpenCV.org, cung cấp một nền tảng cho các nhà phát triển và người dùng¹.
- Năm 2016, Intel mua lại Itseez, một công ty hàng đầu trong lĩnh vực phát triển OpenCV1.
- Năm 2020, phiên bản 4.5.0 của OpenCV được phát hành, hỗ trợ nhiều tính năng mới như ONNX Runtime, Vulkan backend, QR code detector và generator, G-API framework và nhiều hơn nữa.

Tổng quan về OpenCV

- OpenCV được phát hành theo giấy phép BSD, do đó nó hoàn toàn miễn phí cho cả học thuật và thương mại. Nó có các interface C++, C, Python, Java và hỗ trợ Windows, Linux, Mac OS, iOS và Android.
- OpenCV được thiết kế để tính toán hiệu quả và với sự tập trung nhiều vào các ứng dụng thời gian thực. Được viết bằng tối ưu hóa C/C++, thư viện có thể tận dụng lợi thế của xử lý đa lõi.
- Được sử dụng trên khắp thế giới, OpenCV có cộng đồng hơn 47 nghìn người dùng và số lượng download vượt quá 6 triệu lần. Phạm vi sử dụng từ nghệ thuật tương tác, cho đến lĩnh vực khai thác mỏ, bản đồ trên web hoặc công nghệ robot

Nhóm chức năng trong OpenCV

- Xử lý và hiển thị Hình ảnh/ Video/ I/O (core, imgproc, highgui)
- Phát hiện các vật thể (objdetect, features2d, nonfree)
- Geometry-based monocular hoặc stereo computer vision (calib3d, stitching, videostab)
- Computational photography (photo, video, superres)
- Machine learning & clustering (ml, flann)
- CUDA acceleration (gpu)

Các module chính của OpenCV

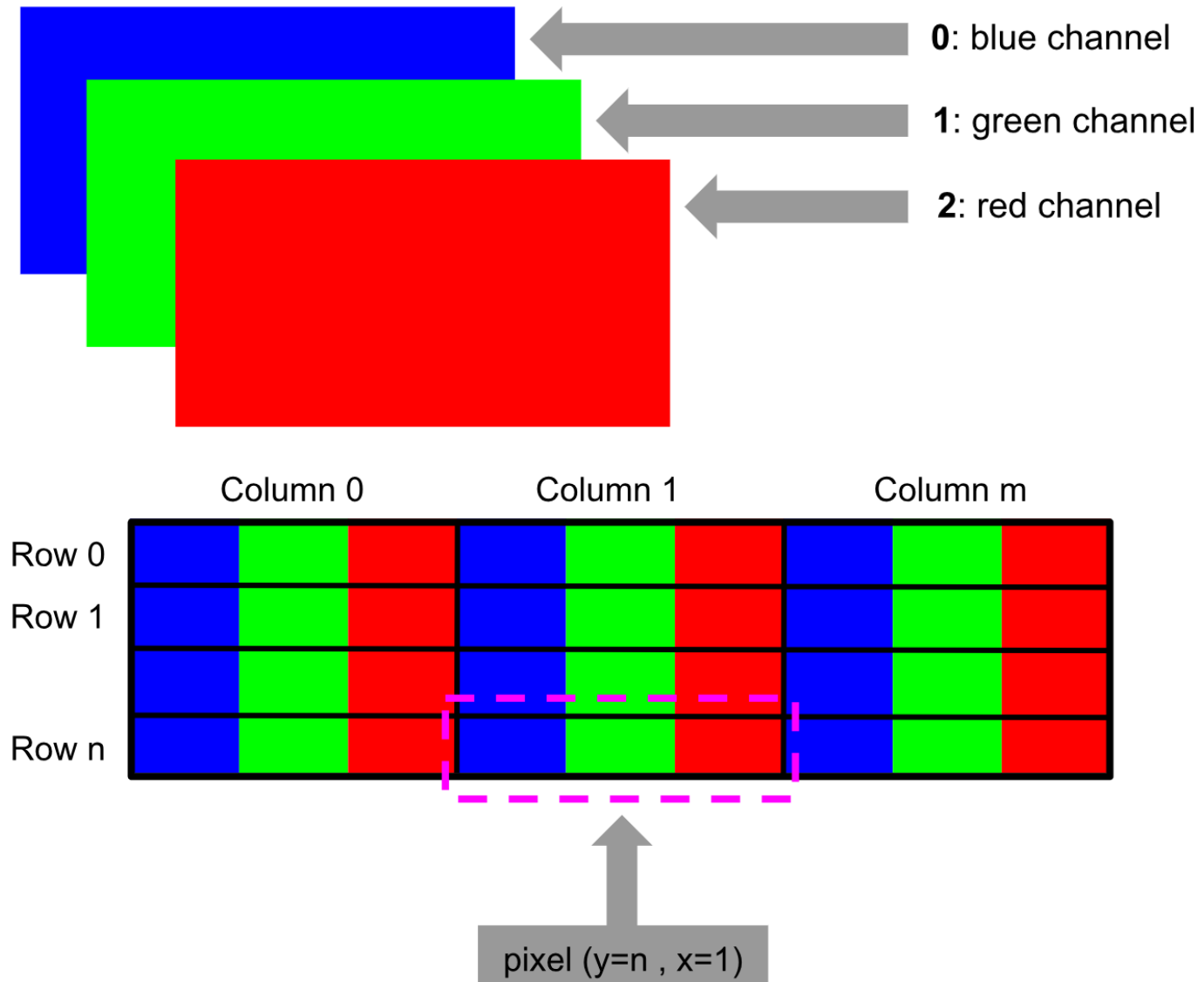
- Core functionality (core) – module nhỏ gọn để xác định cấu trúc dữ liệu cơ bản, bao gồm mảng đa chiều (Mat) và nhiều chức năng cơ bản được sử dụng bởi tất cả các module khác.
- Image Processing (imgproc) – module xử lý hình ảnh gồm cả lọc hình ảnh tuyến tính và phi tuyến (linear and non-linear image filtering), phép biến đổi hình học (chỉnh size, afin và warp phối cảnh, ánh xạ lại dựa trên bảng chung), chuyển đổi không gian màu, biểu đồ, và nhiều cái khác.
- Video Analysis (video) – module phân tích video bao gồm các tính năng ước tính chuyển động, tách nền, và các thuật toán theo dõi vật thể.
- Camera Calibration and 3D Reconstruction (calib3d) – thuật toán hình học đa chiều cơ bản, hiệu chuẩn máy ảnh single và stereo (single and stereo camera calibration), dự đoán kiểu dáng của đối tượng (object pose estimation), thuật toán thư tín âm thanh nổi (stereo correspondence algorithms) và các yếu tố tái tạo 3D.

Các module chính của OpenCV

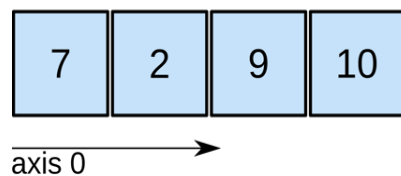
- 2D Features Framework (features2d) – phát hiện các đặc tính nổi bật của bộ nhận diện, bộ truy xuất thông số, thông số đối chọi.
- Object Detection (objdetect) – phát hiện các đối tượng và mô phỏng của các hàm được định nghĩa sẵn – predefined classes (vd: khuôn mặt, mắt, cốc, con người, xe hơi,...).
- High-level GUI (highgui) – giao diện dễ dùng để thực hiện việc giao tiếp UI đơn giản.
- Video I/O (videoio) – giao diện dễ dùng để thu và mã hóa video.
- GPU – Các thuật toán tăng tốc GPU từ các modun OpenCV khác.
- ... và một số module hỗ trợ khác, ví dụ như FLANN và Google test wrapper, Python binding, v.v.

Biểu diễn ảnh màu trong OpenCV

- OpenCV sử dụng không gian màu: **BGR** (blue, green, red)

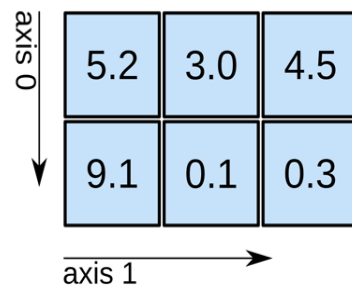


1D array



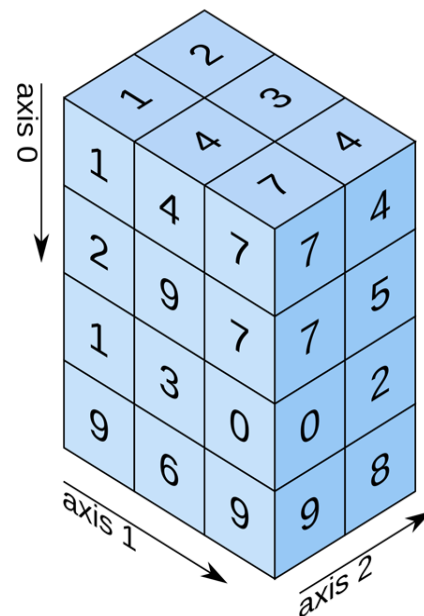
shape: (4,)

2D array



shape: (2, 3)

3D array



shape: (4, 3, 2)

- Khi làm việc với mảng đa chiều, ta sẽ gặp những thuật ngữ sau:
 - Scalar: Mảng 0 chiều (0-D array)
 - Vector: Mảng 1 chiều (1-D array)
 - Matrix: Mảng 2 chiều (2-D array)
 - Tensor: Mảng 3 chiều (3-D array)

numPY - Mảng

Scalar

Vector

Matrix

Tensor

1

$\begin{bmatrix} 1 \\ 2 \end{bmatrix}$

$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$

$\begin{bmatrix} 1 & 2 & 3 & 2 \\ 1 & 7 & 5 & 4 \end{bmatrix}$

■ Khởi tạo mảng

- `arr_1d = np.array([1,2])` # khởi tạo mảng 1 chiều
- `arr_2d = np.array([[1,2],
[3,4]],dtype = int)`
- `arr_3d = np.array([[[1,2],[3,2]],
[[1,7],[5,4]]])`

■ Truy cập phần tử:

- `arr[nrow][ncol]` hoặc `arr[nrow, ncol]`

Thao tác với mảng

- `dtype`: Kiểu dữ liệu của phần tử trong mảng: `float`, `int32`, `int64`, v.v
- `shape`: Kích thước của mảng: (`row`, `col`) với ảnh là (`height:width`) ví dụ (1, 6)
- `size`: Số phần tử trong mảng.
- `ndim`: Số chiều của mảng.
- Ảnh màu trong `numPY` là mảng 3 chiều [`height:width:color_channel`]. Ví dụ ảnh màu có kích thước 800x600 pixels thì:
 - `image.ndim`: số chiều của mảng vì ảnh này là mảng 3 chiều nên kết quả sẽ là 3.
 - `image.shape`: độ lớn các chiều của ảnh bao gồm chiều cao (số dòng, chiều rộng, kết quả là (800,600)
 - `image.dtype`: kiểu dữ liệu của ảnh, kết quả là (`uint8`)

Slicing

- Chúng ta có thể truy cập mảng con theo các chiều thành phần của mảng dựa vào khai báo indices trên từng chiều thông qua khai báo indices của các chiều phân cách bởi dấu phẩy
- Để khai báo indices chúng ta có thể sử dụng slice indices được ngăn cách bởi dấu : Slice indices giúp rút ngắn việc phải nhập từng indices. Nó rất phù hợp với các tình huống các vị trí cần truy cập là liên tục nhau.
- Tổng quát:
 - **start_index : end_index : step** lấy từ **start_index** đến **end_index-1** (không bao gồm end_index) với mỗi bước khoảng cách là **step**
- Bên dưới là 3 công thức slice indice thường sử dụng:
 - **indice_start:indice_end** lấy toàn bộ các indices bắt đầu từ **indice_start** và kết thúc là **indice_end -1**
 - **num_indices** Lấy một num_indices cụ thể.
 - **-num_indices** Lấy một số lượng num_indices ở vị trí cuối cùng.
 - **:num_indices** Lấy một số lượng num_indices ở vị trí đầu tiên.
 - **:** (chỉ có dấu 2 chấm) là tất cả trên chiều đó

Slicing

data

(0, 0)	(0, 1)	(0, 2)	(0, 3)
(1, 0)	(1, 1)	(1, 2)	(1, 3)
(2, 0)	(2, 1)	(2, 2)	(2, 3)
(3, 0)	(3, 1)	(3, 2)	(3, 3)

data[0]

(0, 0)	(0, 1)	(0, 2)	(0, 3)
(1, 0)	(1, 1)	(1, 2)	(1, 3)
(2, 0)	(2, 1)	(2, 2)	(2, 3)
(3, 0)	(3, 1)	(3, 2)	(3, 3)

data[1, :]

(0, 0)	(0, 1)	(0, 2)	(0, 3)
(1, 0)	(1, 1)	(1, 2)	(1, 3)
(2, 0)	(2, 1)	(2, 2)	(2, 3)
(3, 0)	(3, 1)	(3, 2)	(3, 3)

data[:, 2]

(0, 0)	(0, 1)	(0, 2)	(0, 3)
(1, 0)	(1, 1)	(1, 2)	(1, 3)
(2, 0)	(2, 1)	(2, 2)	(2, 3)
(3, 0)	(3, 1)	(3, 2)	(3, 3)

data[0:2, 0:2]

(0, 0)	(0, 1)	(0, 2)	(0, 3)
(1, 0)	(1, 1)	(1, 2)	(1, 3)
(2, 0)	(2, 1)	(2, 2)	(2, 3)
(3, 0)	(3, 1)	(3, 2)	(3, 3)

data[0:2, 2:4]

(0, 0)	(0, 1)	(0, 2)	(0, 3)
(1, 0)	(1, 1)	(1, 2)	(1, 3)
(2, 0)	(2, 1)	(2, 2)	(2, 3)
(3, 0)	(3, 1)	(3, 2)	(3, 3)

data[:, ::2]

(0, 0)	(0, 1)	(0, 2)	(0, 3)
(1, 0)	(1, 1)	(1, 2)	(1, 3)
(2, 0)	(2, 1)	(2, 2)	(2, 3)
(3, 0)	(3, 1)	(3, 2)	(3, 3)

data[1::2, 1::2]

(0, 0)	(0, 1)	(0, 2)	(0, 3)
(1, 0)	(1, 1)	(1, 2)	(1, 3)
(2, 0)	(2, 1)	(2, 2)	(2, 3)
(3, 0)	(3, 1)	(3, 2)	(3, 3)

Một số phép tính thường dùng

- Các phép cộng, trừ, nhân (+ - *) 2 ma trận có cùng kích thước theo phương pháp lấy từng phần tử của ma trận 1 tính toán với phần tử tương ứng của ma trận thứ 2.
- Các phép cộng, trừ, nhân (+ - *) ma trận với 1 số theo phương pháp lấy từng phần tử của ma trận tính toán với số đó
- Ví dụ chúng ta có thể cộng hai hai ma trận có cùng kích :

$$\mathbf{A}_{(m \times n)} + \mathbf{B}_{(m \times n)} = \begin{bmatrix} a_{11} + b_{11} & a_{12} + b_{12} & \cdots & a_{1n} + b_{1n} \\ a_{21} + b_{21} & a_{22} + b_{22} & \cdots & a_{2n} + b_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} + b_{m1} & a_{m2} + b_{m2} & \cdots & a_{mn} + b_{mn} \end{bmatrix}$$

Một số hàm trên numpy

- `arr.max()` hoặc `numpy.max(arr)`: Lấy giá trị lớn nhất của mảng `arr`.
- `arr.min()` hoặc `numpy.min(arr)`: Lấy giá trị nhỏ nhất của mảng `arr`.
- `arr.sum()` hoặc `numpy.sum(arr)`: Tổng tất cả các phần tử trong mảng `arr`.
- `arr.mean()` hoặc `numpy.mean(arr)`: Trung bình cộng của tất cả các phần tử trong mảng `arr`.
- `numpy.median(arr)`: Trả về giá trị trung vị của mảng `arr`.

Ví dụ về numpy, OpenCV trong python

- Load, show, crop, save, color split
- Lib: numpy, opencv-contrib-python

